

QMC Option Pricing

A repository on the implementation of QMC option pricing techniques under Black Scholes.

Pablo Ortiz Badenes

June 1, 2026

Contents

1	Introduction	1
1.1	Preface	1
1.2	Introduction	1
1.3	The Black-Scholes model	2
2	Monte Carlo	3
2.1	Monte Carlo implementation	3
2.2	Monte Carlo convergence	3
3	Quasi Monte Carlo	6
3.1	Antithetic variates	6
3.2	Stratified sampling	7
3.3	Low discrepancy sequences	8
3.3.1	Kronecker	8
3.3.2	Halton (Van der Corput)	9
3.3.3	Sobol	9
3.3.4	Numerical comparison	10
3.4	Low discrepancy sequences applied to Black-Scholes	11
4	Importance sampling	13
4.1	Naive importance sampling	13
4.2	Cross Entropy importance sampling	15
5	Path dependent options	17
5.1	Monte Carlo implementation	17
5.2	Antithetic variates	18
5.3	Control variates	18
6	Final findings	20

List of Figures

2.1	Monte Carlo Convergence rate	4
2.2	Monte Carlo performance metrics	5
3.1	Antithetic variates performance metrics	6
3.2	Antithetic variates vs Monte Carlo	7
3.3	Stratified sampling metrics	7
3.4	Antithetic variates vs Stratified sampling	8
3.5	Computation time vs Variance	10
3.6	Kronecker sampling metrics	11
3.7	Halton sampling metrics	11
3.8	Sobol sampling metrics	12
3.9	Sobol vs Stratified sampling	12
4.1	Monte Carlo vs Importance Sampling variance	14
4.2	Monte Carlo vs Importance Sampling relative variance	14
4.3	Cross-entropy vs Importance sampling relative variance	16
5.1	Monte Carlo Black Scholes vs Path Dependent volatility	17
5.2	Path dependent MC vs Antithetic variates	18
5.3	Path dependent MC vs Antithetic variates	18
5.4	Path dependent MC vs Control Variates	19

1 Introduction

1.1 Preface

This repository is meant mostly as a learning project on low discrepancy sampling and importance sampling, as well as its computational implementation. To apply it to real world scenarios, the classic example of European options pricing under Black-Scholes hypothesis is taken, as it is a foundational benchmark for these methods.

The goal is to provide an incremental approach, starting from the analytical solution to the Black-Scholes model, as well as its Monte Carlo computational implementation, to then extend it to various Quasi Monte Carlo techniques, as well as variance reduction and importance sampling techniques. Further along, the hypothesis and assumptions of Black Scholes will be changed and made more strict, to get more complex, but more realistic, models.

Most of the Python implementations will use the functions given by the NumPy and SciPy modules, due to their stability and speed. Nonetheless, the reasoning behind the techniques here used are applicable to any language, as well as implementations from scratch. Unless otherwise stated, rounding error, as well as error attributed to the NumPy functions (number generation not being perfectly random or normal, inverse cdf having slight error,...) will be neglected, so exact values found through closed form solutions will be considered perfectly exact, and inconsistencies will not be attributed to machine precision. This is clearly an assumption, but generally, the magnitude of error will be considerably bigger than machine precision, making this assumption valid.

1.2 Introduction

European options are a type of financial derivative that give the holder the right, but not the obligation, to buy or sell an underlying asset at a predetermined strike price on a specific expiration date. A European call option allows the purchase of the asset at maturity, while a European put option allows its sale. Unlike American options, European options can only be exercised at expiration, which makes them easier to analyze mathematically.

The Black-Scholes model is one of the most important frameworks for pricing European options. Developed by Fischer Black, Myron Scholes, and Robert Merton in the early 1970s, the model provides a closed-form mathematical formula for determining the theoretical value of European call and put options. The model assumes that markets are efficient and that investors can continuously hedge risk by dynamically adjusting positions in the underlying asset. Because European options have a fixed exercise date, the Black-Scholes framework applies particularly well to them.

The Black-Scholes model relies on several simplificative hypotheses. First, it assumes that the price of the underlying asset follows a continuous stochastic process known as geometric Brownian motion, implying that returns are normally distributed and volatility remains constant over time. It also assumes frictionless markets, meaning there are no transaction costs, taxes,

or restrictions on short selling. Additionally, the model considers a constant risk-free interest rate and assumes that trading can occur continuously. Another key assumption is that the underlying asset does not pay dividends during the option's life, although later extensions of the model incorporate dividend payments.

1.3 The Black-Scholes model

The Black-Scholes model determines the fair price for an option (premium), in order to assume no expected net gain after its maturity. It works under the aforementioned assumptions. Its mathematical expression is

$$C(S_0, K, T, r, \sigma) = S_0 \Phi(d_1) - K e^{-rT} \Phi(d_2)$$

where

$$d_1 = \frac{\ln\left(\frac{S_0}{K}\right) + \left(r + \frac{1}{2}\sigma^2\right) T}{\sigma\sqrt{T}}$$

and

$$d_2 = d_1 - \sigma\sqrt{T}$$

or equivalently,

$$d_2 = \frac{\ln\left(\frac{S_0}{K}\right) + \left(r - \frac{1}{2}\sigma^2\right) T}{\sigma\sqrt{T}}$$

Its parameters are

- S_0 : current stock price
- K : strike price
- T : time to maturity (in years)
- r : continuously compounded risk-free interest rate
- σ : volatility of the underlying asset
- $\Phi(\cdot)$: cumulative distribution function (CDF) of the standard normal distribution

This is the closed formula for the Black-Scholes model, and will be used as a benchmark for all the following methods. This way, these methods can be compared on a problem with known solution, and then extrapolated to other assumptions or more exotic options.

This benchmark function is implemented inside the `Benchmark.py` file, under the name `european_call_cf(S0,K,T,r,sigma)`.

2 Monte Carlo

2.1 Monte Carlo implementation

One of the assumptions for Black-Scholes is that stock prices follow Geometric Brownian Movement. Thus, given the same parameters as the previous model, one could simulate several possible paths for the stocks, and decide at maturity, given the stock price and strike price, whether or not it will be profitable to execute the option, as well as the payoff. Taking into consideration the depreciation of assets due to the risk free interest rate, one can approximate the fair price for an option. The additional parameter needed to implement this procedure is the number of iterations, this is, the number of stock paths simulated.

When normally distributed random numbers are used, it is called Monte Carlo simulation, and it is the simplest computational implementation of Black-Scholes. This function is implemented inside `MC_BlackScholes.py` as `european_call_mc(S0, K, T, r, sigma, n)`. The program returns the calculated value for the option price. This file also contains the Monte Carlo expected return, `expected_profit_mc(S0,K,T,r,sigma,option_price,n)`, which, using the same logic, takes option price as an input, simulates n stock prices, and returns the expected profit over n iterations.

By default, the programs run a single time, but an additional argument `exp` can be added to run various experiments with the same parameters, and return a vector of experiments. Vectorizing the program twice (samples and experiments) is faster than using for loops for experiments, as numpy runs on C, so making less calls to the Python interpreter, while inputting bigger matrices (rather than vectors) is faster. However, for really big batches, RAM will become a limitation faster with this double vectorization, as the whole matrix will be bigger than running each vector separately.

For values close to the ones given by `european_call_mc`, it will return values close to 0, as the option price obtained is the fair price. For lower option prices, expected values will generally be positive, and for higher option prices, they will generally be negative.

2.2 Monte Carlo convergence

One of the aspects that will be improved on along the following implementations will be the convergence rate. Given a theoretical exact value $\bar{x}$ and the sequence of Monte Carlo approximations $\{x_n\}_n$ using n points, the following is true,

$$\lim_n |\bar{x} - x_n| = 0$$

Thus, the Monte Carlo method will provide increasingly good approximations. Nonetheless, the rate of convergence will be of the order of $\frac{1}{\sqrt{n}}$, so the method has an order of convergence $\mathcal{O}(n^{-\frac{1}{2}})$.

Let

$$E_n \equiv |\bar{x} - x_n| = \frac{C}{\sqrt{n}}$$

for some value of $C \in \mathbb{R}$. Then, $\log E_n = -\frac{1}{2} \log n + \log C$. Therefore, the log-log plot of the error versus the sample size should be linear. For some given parameters (detailed in `MonteCarlo_Convergence.ipynb`), the Monte Carlo error rate is represented for certain values of N (sample size), taking the average over 100 experiments. The 95% confidence interval bars are also shown.

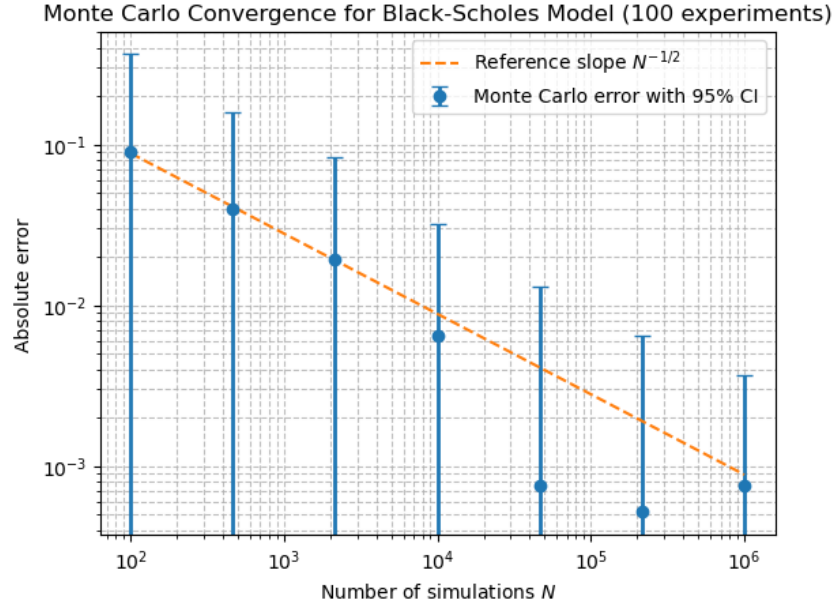


Figure 2.1: Monte Carlo Convergence rate

Additionally, the three defining metrics that will be used on these methods can be introduced. These are the Bias, variance and RMSE (root mean squared error).

Bias measures the systematic error introduced by approximating a real-world problem with a simplified model. It reflects how far the average prediction is from the true value.

$$\text{Bias}(x) = \mathbb{E}[x] - \bar{x}$$

Variance measures how much the model's predictions fluctuate when trained on different datasets. It reflects model sensitivity to data noise.

$$\text{Variance}(x) = \mathbb{E}[(x - \mathbb{E}[x])^2]$$

RMSE measures the average magnitude of prediction errors, giving higher weight to large errors. It combines both bias and variance effects.

$$\text{RMSE} = \sqrt{\mathbb{E}[(\hat{x} - y)^2]}$$

Generally,

- High bias $\rightarrow$ model is too simple (underfitting)
- Low bias $\rightarrow$ model closely captures the true relationship
- High variance $\rightarrow$ model is too complex (overfitting)
- Low variance $\rightarrow$ model is more stable across datasets
- Lower RMSE $\rightarrow$ better predictive accuracy (has the same units as the predicted magnitude)

Due to the fact that $\text{RMSE}^2 \approx \text{Variance} + \text{Bias}^2$, generally Bias^2 will be plotted over Bias.

For this implementation using Monte Carlo simulation, these are the results obtained, as well as its fitting lines

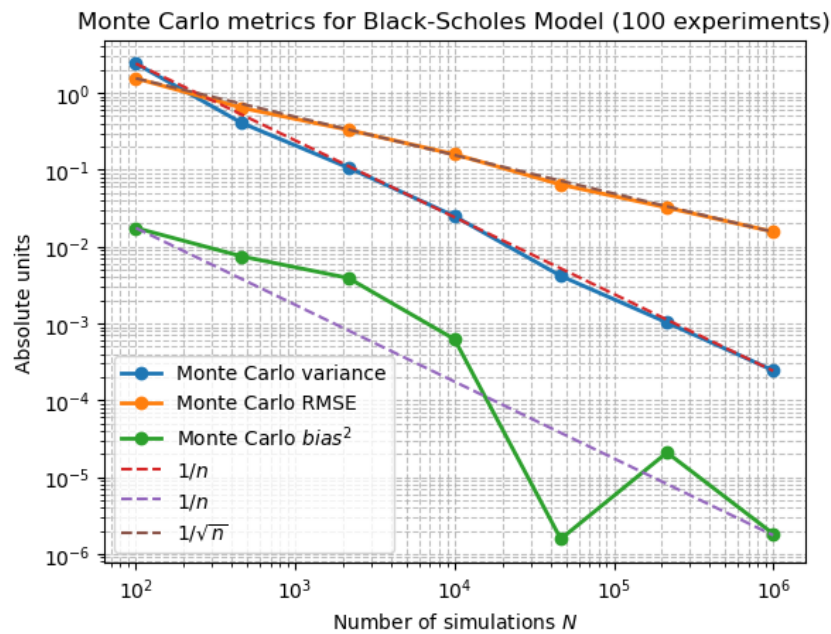


Figure 2.2: Monte Carlo performance metrics

(note how Bias^2 fits to $1/n$, as $E_n \approx 1/\sqrt{n}$)

3 Quasi Monte Carlo

3.1 Antithetic variates

Once the baseline Monte Carlo implementation is done and tested, the object of this repository can be expanded on. The goal now is to test various techniques that can be used to obtain Quasi Monte Carlo (QMC) simulations that reduce the variance over plain Monte Carlo, leading to better results on smaller sample sizes. The first one that will be implemented is the use of antithetic variates. The idea behind it is to generate a sample size Z of $\frac{n}{2}$ samples instead of n , but simulate both the Z and $-Z$ paths. This reduces the computational cost of the sample generation, and can noticeably reduce bias and variance on well behaved problems. However, it can break independence of samples, and distort estimates on certain problems. It is implemented as `european_call_at(S0,K,T,r,sigma,n)`, also having `exp` as an additional argument. With the same values as before, the metrics for antithetic variates are found. However, instead of assuming $\frac{C}{\sqrt{n}}$ or $\frac{C}{n}$ for convergence, the least squares approximation for the log-log plot is found (using an auxiliary function).

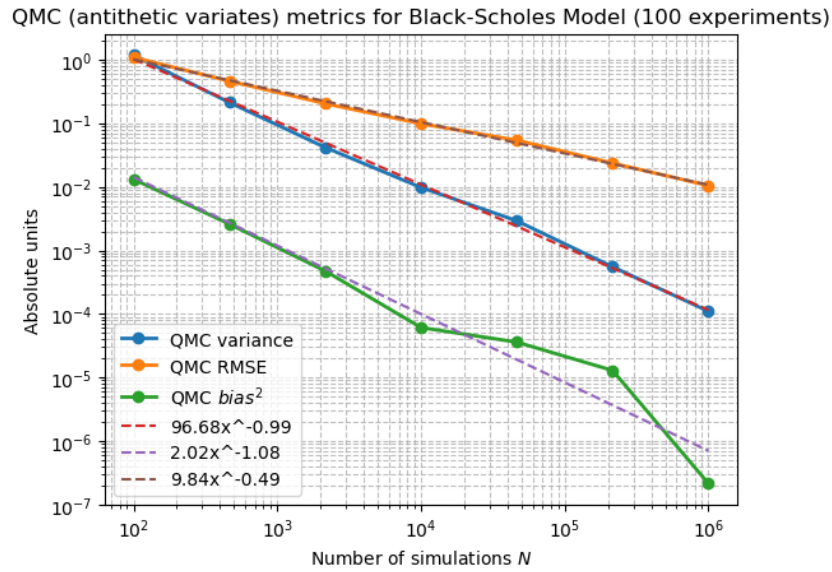


Figure 3.1: Antithetic variates performance metrics

Furthermore, the computation time and variance of Monte Carlo and antithetic variates can be compared. As expected, antithetic variates reduces both the variance and computation time. Furthermore, it reduces the variance by a factor of approximately 2.

Computation time versus Variance for Black-Scholes Model (100 experiments)

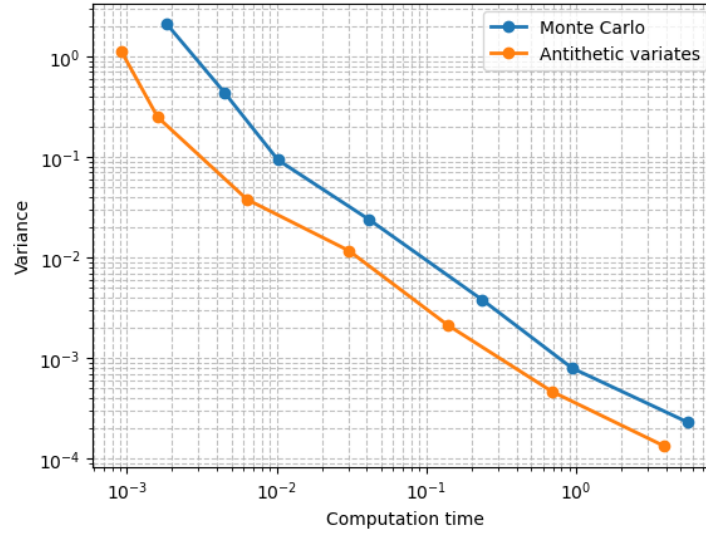


Figure 3.2: Antithetic variates vs Monte Carlo

3.2 Stratified sampling

The next Quasi Monte Carlo technique that will be implemented is stratified sampling. From this point onwards, the point generation will happen in $U[0, 1]$, due to the fact that most of these techniques are optimized for this interval, and that uniform points in $[0, 1]$ can be mapped to Gaussian sampling in $\mathbb{R}$ by using the inverse continuous density function (however, all functions in the repository are implemented using `norm.ppf` to make compute benchmarks fair). The reasoning behind stratified sampling is that random sampling, despite being uniform, leaves gaps, as well as point clusters. By dividing the interval into smaller chunks, and generating the corresponding fraction of the points in it, the point distributions should be more uniform. Due to the increase in function calls, as well as the added compute logic, there should be a higher computational cost, however the experimental results show the tradeoff is worth it.

QMC (stratified sampling) metrics for Black-Scholes Model (100 experiments)

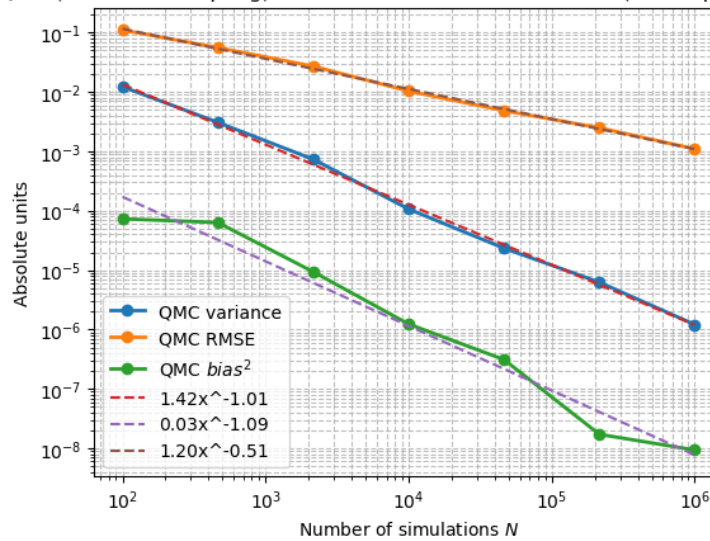


Figure 3.3: Stratified sampling metrics

The metrics fit to the expected values, and when compared to antithetic variates, a clear reduction in variance is found, at the expense of a slight increase in computational cost.

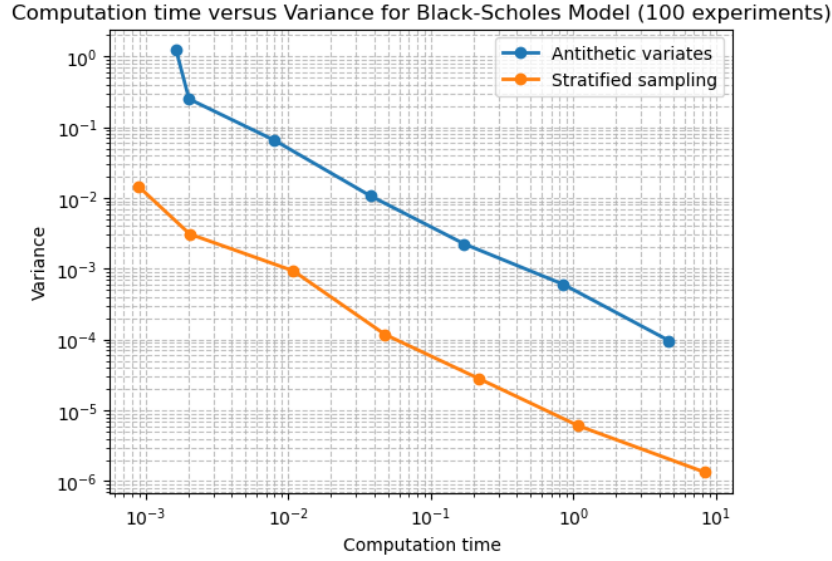


Figure 3.4: Antithetic variates vs Stratified sampling

In this case, the variance reduction (for the same n) averages to be around an 85 times decrease, returning a variance nearly two orders of magnitude smaller, while approximately duplicating compute time, when compared to stratified sampling.

3.3 Low discrepancy sequences

This section aims to quickly explain the low discrepancy sequences that will then be implemented into the QMC Black Scholes model. The idea behind low discrepancy sequences is using deterministic but apparently random sequences that cover space more efficiently than pure random numbers. This way, gaps become smaller and variance between samples is reduced.

3.3.1 Kronecker

Kronecker sequences are generated by taking the fractionary part of multiples of irrational numbers. When numbers that are badly approximated by rational numbers are taken, the fractionary parts of these multiples cover the unit interval with low discrepancy. Mathematically, let α be irrational, the sequence is defined by

$$\{x_n\}_n = (\alpha n) \mod 1$$

Due to requiring only multiplication and modular arithmetic, they can be easily generated by a computer. For this implementation, the golden ratio conjugate $\alpha = \frac{\sqrt{5}-1}{2}$ is used. The main drawback of this method is that repetitions of the same experiment will yield the same results, if α is taken as constant. This can be solved by changing bases, scrambling the sequence, or skipping numbers, but this will not be discussed in this repository. When n experiments of m samples each are needed, a $m \cdot n$ long vector will be generated and reshaped.

3.3.2 Halton (Van der Corput)

The Halton sequence generalises the one-dimensional Van der Corput sequence to multiple dimensions by assigning a different coprime base to each coordinate. This implementation focuses on the 1-dimensional case. The Van der Corput sequence in base b is constructed by writing the integer index n in base b , reversing its digits, and interpreting the result as a fraction in $[0, 1)$:

$$\phi_b(n) = \sum_{k=0}^{L-1} d_k(n) b^{-k-1},$$

where

$$n = \sum_{k=0}^{L-1} d_k(n) b^k$$

Because only one base is used, this routine generates a one-dimensional Halton (Van der Corput) sequence. For multi-dimensional Halton sequences one would need different coprime bases per dimension, which is not implemented here. The scrambling allows for m independent experiments to be generated while preserving low discrepancy.

3.3.3 Sobol

Sobol sequences are a family of quasi-random sequences that achieve very low discrepancy by using direction numbers and a Gray code construction. In essence, the n -th Sobol point in d dimensions is obtained by XOR-ing together direction vectors selected according to the binary representation of n . The implementation provided generates a one-dimensional Sobol sequence (extendable to multiple dimensions by using different direction numbers per coordinate).

The algorithm, designed for efficiency, works with 32-bit unsigned integers:

1. A `skip` parameter (default 100) discards initial points, which can improve uniformity by avoiding early transient behaviour.
2. Indices $i = \text{skip}, \text{skip} + 1, \dots, \text{skip} + m \times n - 1$ are generated.
3. For each index, the number of trailing zeros (`tz`) is computed using `(i) & (i+1)`. The position of the rightmost zero bit determines which direction number to use.
4. The direction numbers `dirs` are precomputed as `1<<(bits-c)` where `c=floor(log2(tz))+1`. This corresponds to a simple set of primitive polynomials (the Sobol sequence in base 2).
5. A cumulative XOR over the direction numbers yields the integer Sobol values `x`.
6. These integers are scaled to the unit interval by dividing by $2^{\text{bits}} = 2^{32}$.

The resulting vector is reshaped into an $m \times n$ array. The implementation is fast because it relies on bitwise operations and avoids explicit Gray code indexing.

3.3.4 Numerical comparison

These methods are implemented as `Kronecker(m,n)`, `Halton(m,n)` and `Sobol(m,n)`. When compared to the default random number generation for numpy, a significant improvement in variance can be found, at a higher computational cost. As detailed in `Sequence_testing.ipynb`, the improvement in variance is of approximately two orders of magnitude. When comparing variance to computational cost (independent of number of samples), only this implementation of Sobol matches the random number generation. Nonetheless, in the next section, numpy implementations for these sequences will be used, so computational cost will be improved on further.

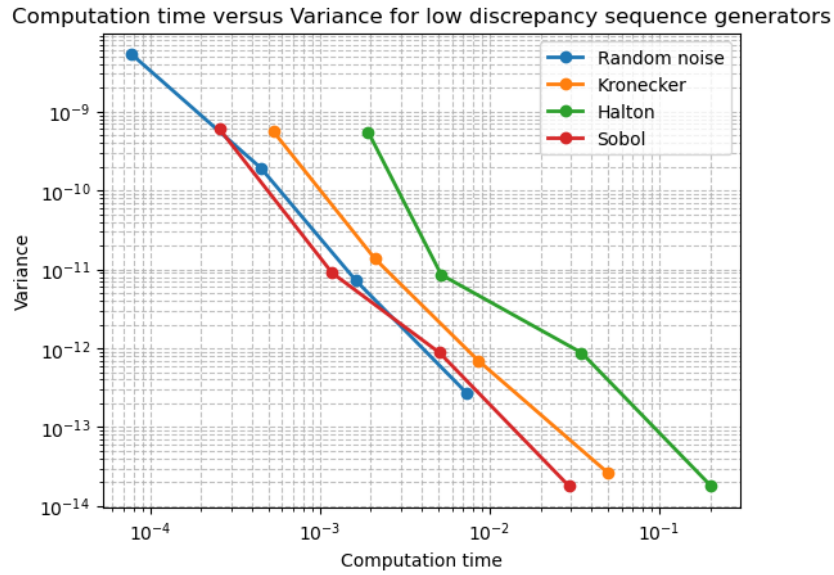


Figure 3.5: Computation time vs Variance

3.4 Low discrepancy sequences applied to Black-Scholes

These low discrepancy sequences can be applied to the Black Scholes simulation used on previous sections, and when tested on the same parameters, a clear reduction in variance (both absolute and variance reduction rate) can be seen. The test results are the following

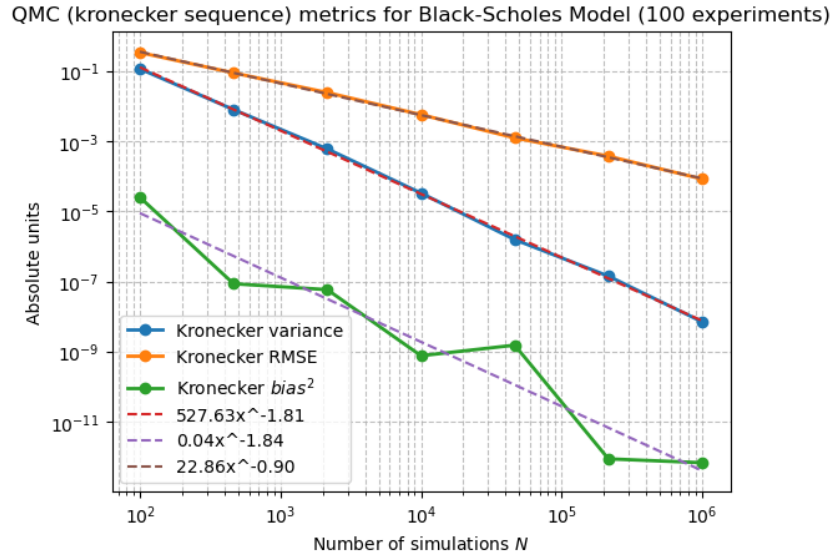


Figure 3.6: Kronecker sampling metrics

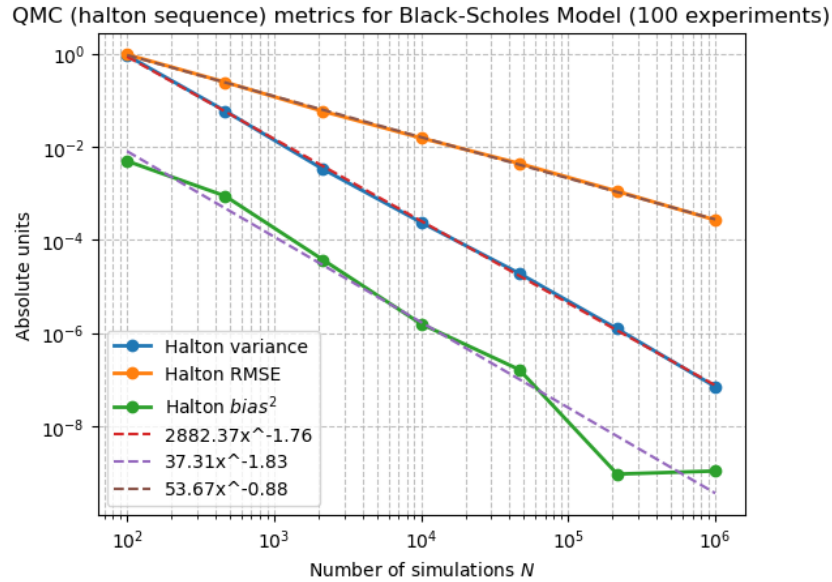


Figure 3.7: Halton sampling metrics

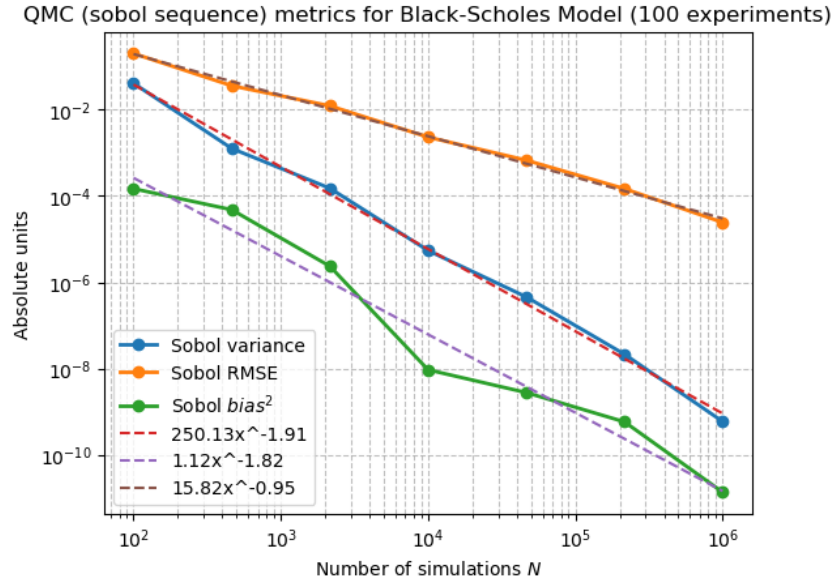


Figure 3.8: Sobol sampling metrics

These methods show a variance rate of $\mathcal{O}(n^{-\alpha})$, being α between 1.5 and 2, compared to the value of 1 found for antithetic variates and stratified sampling. Due to Sobol being the best performing sequence out of the three candidates, it can be tested against stratified sampling, the previous best performing method.

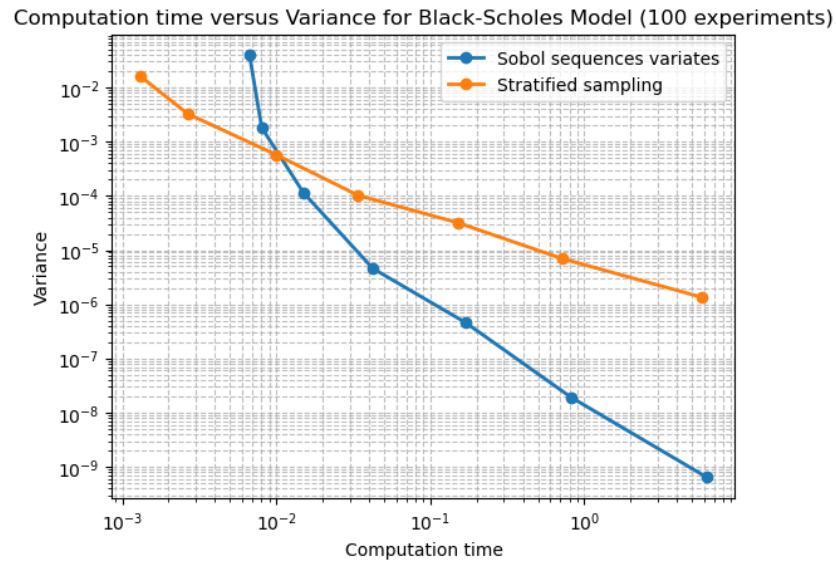


Figure 3.9: Sobol vs Stratified sampling

As it can be seen on the plot (more plots can be found in `QMC_Convergence.ipynb`), Sobol sequences have a higher computational cost over stratified sampling across the board. However, the faster asymptotic variance reduction rate of Sobol sequences make up for the differences for bigger sample sets, making it theoretically more suitable for real world applications.

4 Importance sampling

4.1 Naive importance sampling

One of the the main limitations of the mehtods previously shown is that it correlates the regions with the highest probability to the ones with the highest payout. For strike prices K close to the stock price S_0 , a high portion of all estimated terminal prices will provide a payout. However, as the difference between stock price and strike prices grows bigger (either by having $K \ll S_0$ or $K \gg S_0$), an increasingly smaller amount of all simulated paths will return a payout (most will return 0). This reduction in simulated paths that yield payout will increase the variance, while maintaining the same computational cost. Thus, the current methods are not suitable for extreme situations that would provide high payouts, such as when K is much bigger than S_0 (deep in the money).

This mismatch caused by the majority of the probabilities being close to the original stock price, while the payout only occuring when the stock price passes the strike price, can be solved with importance sampling. All the stock paths between the original stock price and the strike price provide exactly 0 payout (and 0 option price). Thus, instead of using a gaussian distribution with mean 0, the mean can be moved closer to the strike price, and the resulting paths can then be adjusted to account for the low likelihood of those paths occuring. Bayes's theorem states

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

In this context, the zero mean gaussian can be calculated using a mean shifted gaussian, and a likelihood function to adjust the weight. Let $p(x) = N(0, 1)$, when sampling from $q(x) = N(a, 1)$, a function $L(x)$ is needed such that $p(x) = q(x)L(x)$. Substituting in the expressions for the gaussian distribution,

$$\frac{1}{\sqrt{2\pi}}e^{-x^2/2} = L(x)\frac{1}{\sqrt{2\pi}}e^{-(x-a)^2/2} \implies L(x) = e^{-ax+a^2/2}$$

This can be easily implemented onto the Monte Carlo Black Scholes programs (as well as any of the QMC variants). For simplicity's sake, it will be implemented in and tested against base Monte Carlo. It is implemented as `european_call_is(S0, K, T, r, sigma, a, n, exp)`, and tested more thoroughly on `IS_Convergence.ipynb`. The shift used must consider the fact that growth is considered to be exponential, and the effect the time and volatility will have. The expression used in this section will be $a = \ln(\frac{K}{S_0})\frac{1}{\sigma\sqrt{T}}$. When using this expresison, the main findings are that error is reduced, but more notably that variance can be greatly reduced. Furthermore, if strike prices sufficiently high were chosen, standard Monte Carlo might fail to get even one sample into the payout region, incorrectly returning both 0 for the price and variance. This gaussian mean shift eliminates that chance. The variance can be compared against the distance to the strike price.

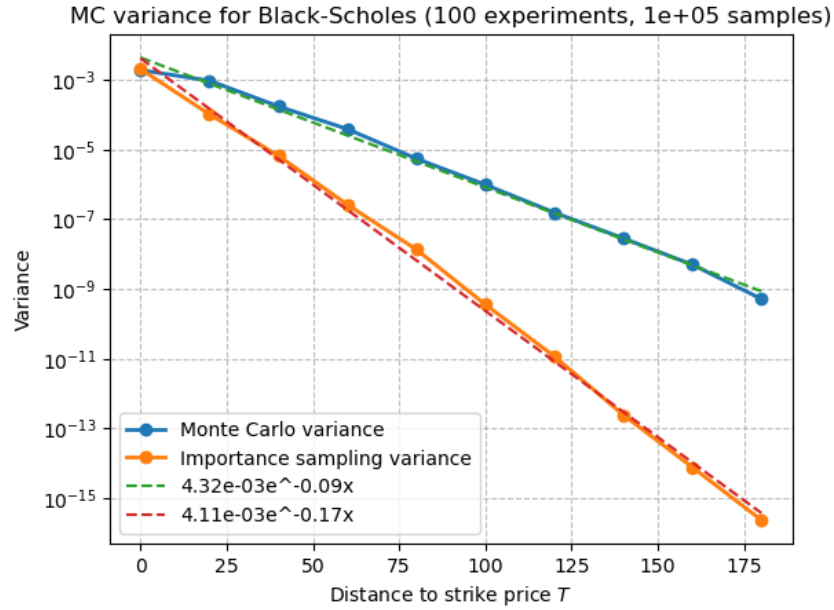


Figure 4.1: Monte Carlo vs Importance Sampling variance

The plot shows both variances decreasing, with Importance sampling having smaller variances. However, this reduction in both variances is due to the option price also getting smaller. If $\frac{\text{Variance}}{\text{Price}^2}$ is plotted instead (relative variance, square used so units match), the true benefit of importance sampling can be seen.

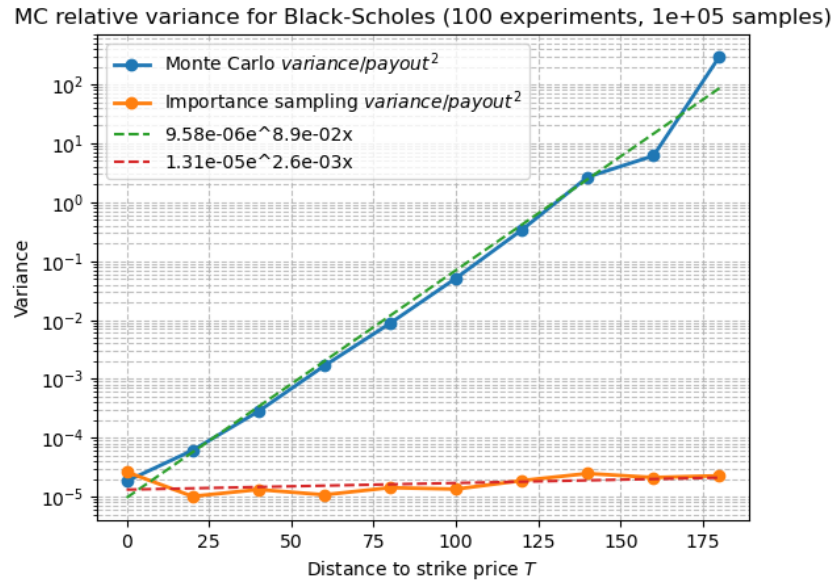


Figure 4.2: Monte Carlo vs Importance Sampling relative variance

This plot clearly shows how Importance sampling allows for relative variance to remain more or less constant despite rare events, while Monte Carlo's variance collapses.

4.2 Cross Entropy importance sampling

One of the main drawbacks of this importance sampling implementation is that it relies heavily on taking a good estimate for the mean shift. For simple models, such as the standard European Black Scholes option pricing, an estimate can be deduced off the models dynamics, and a closed expression can be derived and proved to be optimal. In this case, the optimal mean shift is

$$a^* = \frac{\ln(\frac{K}{S_0}) - (r - \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}$$

However, for problems where this optimal value cannot be derived analytically, other techniques must be used. The adaptive importance sampling technique that will be explored here is Cross-entropy. Theoretically, a distribution exists, such that variance is exactly zero for the simulations of the model. Cross-entropy simplifies the problem by restricting it to the family $N(\mu, 1), \mu \in \mathbb{R}$, and choosing the closest (according to Kullback-Liebler divergence) element from that family. The Kullback-Liebler divergence between a true probability distribution P and a distribution Q is defined as:

$$D_{KL}(P||Q) = \sum_{x \in X} P(x) \log \frac{P(x)}{Q(x)}$$

Let $H(S_T)$ be the payout associated to a stock price at maturity S_T , and $Z \equiv Z(0, 1)$. Then, it can be derived (proof is omitted) that the optimal mean shift will satisfy:

$$a^* = \frac{\mathbb{E}[H(Z)Z]}{\mathbb{E}[H(Z)]}$$

An iterative algorithm to implement this is:

1. Set $a_0 = 0$
2. Compute $w_i = H(Z_i)L(Z_i)$
3. Compute $a_{k+1} = \frac{\sum w_i Z_i}{\sum w_i}$
4. Repeat using the new mean shift

In this repository's implementation, a damping factor is added to mitigate possible numerical instability. When compared to the previous naive approximation for the mean shift, it can be seen that it reduces variance, due to it getting close to the true optimal mean shift. It is noticeably noisier than that naive estimate, due to the relatively low max iterations number used for compute time purposes. However, this shows that, with sufficiently high numbers of samples, experiments and maximum iterations, the Cross-entropy method would converge towards the true optimal mean shift. Furthermore, in problems where no closed form solution exists, it can be a way to provide much more reliable results than blindly estimating.

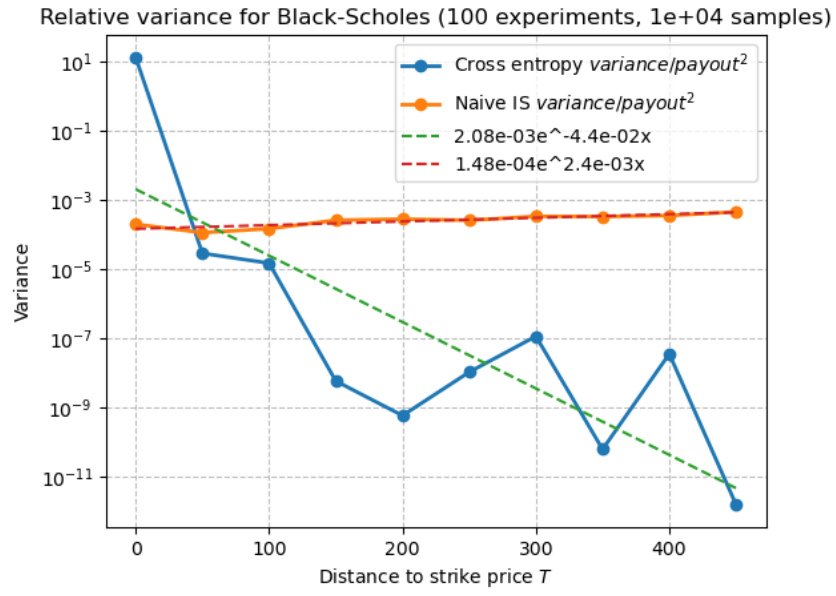


Figure 4.3: Cross-entropy vs Importance sampling relative variance

5 Path dependent options

5.1 Monte Carlo implementation

This section focuses on path dependent options, which refers to options where the evolution of the stock price has to be computed. This section isn't by no means exhaustive, as this topic is extremely deep and complex. It aims, however, to provide some basic insight on the paradigm changes and structural adaptations needed to move onto path dependent options. The standard Black Scholes assumes, among others, constant volatility. Thus, a single random value can be generated for each sample. In the following methods, volatility will not be constant, but rather follow a stochastic process given by

$$\sigma = \rho Z + \sqrt{1 - \rho^2} \cdot U, \quad \rho \in [0, 1], \quad Z, U \sim N(0, 1)$$

with Z being the stochastic variable that models price changes. The main drawback of path dependent options is the increase in computational cost, due to there being an extra dimension. For n samples and e experiments, the cost of a Monte Carlo Black Scholes computation is $\mathcal{O}(n \cdot e)$, however, for a path dependent option using t timesteps, the computational cost will be roughly $\mathcal{O}(n \cdot e \cdot t)$. Thus, variance reduction techniques gain importance, as they can provide similar levels of accuracy with much smaller sample sizes, speeding up computation.

This path dependent model is implemented using Monte Carlo simulation as a baseline, as `path_dependent_mc(S0, K, T, r, rho, timesteps, n, exp)`. When compared to the regular Monte Carlo Black Scholes implementation, it can be noted that variance is much higher, due to the added randomness of volatility following a stochastic process. The asymptotic rate, however, is the same.

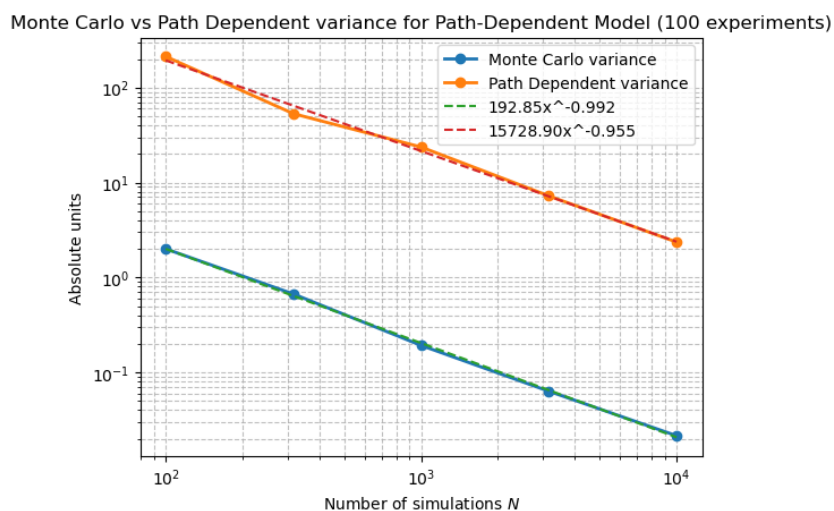


Figure 5.1: Monte Carlo Black Scholes vs Path Dependent volatility

5.2 Antithetic variates

The simplest improvement to Monte Carlo, also for path dependent options, is the use of antithetic variates. By simulating a path and its opposite, the computational cost of number generation is reduced, and variance is also decreased. It is implemented as `path_dependent_at(S0, K, T, r, rho, timesteps, n, exp)`.

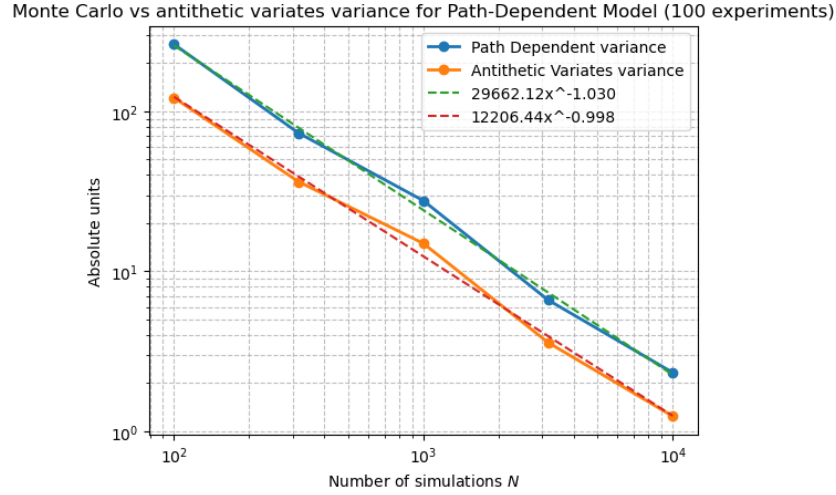


Figure 5.2: Path dependent MC vs Antithetic variates

Note how computation time is also lower, on top of variance being lower.

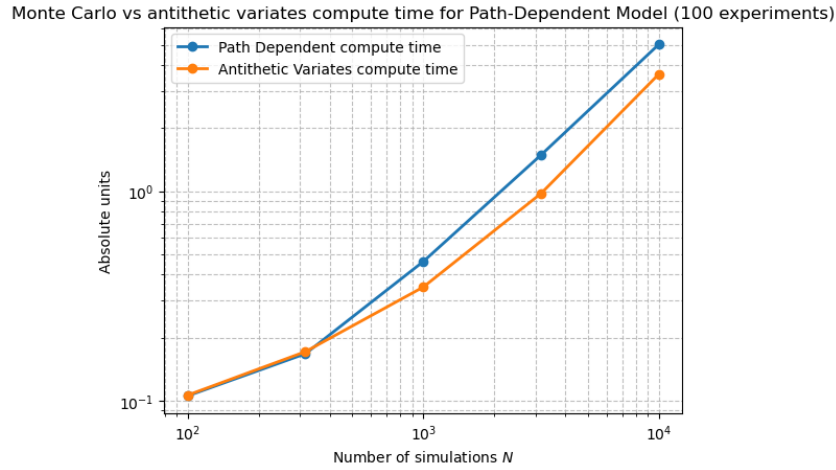


Figure 5.3: Path dependent MC vs Antithetic variates

5.3 Control variates

Finally, control variates can be briefly explained. Control variates are a really powerful variance reduction technique, where an estimated measure is corrected using a known and easily computable control metric. In this case, the closed form solution for european options under Black Scholes is used to correct the previously described random volatility model. The option price under stochastic volatility will be X

$$X = e^{-rT} \max(S_T - K, 0)$$

So the Monte Carlo estimator is $\mathbb{E}[X]$. A second variable Y (Black Scholes closed form solutions), that is highly correlated with X is introduced, as $\mathbb{E}[Y]$ easy to compute. Thus, the simulated value X is corrected using

$$X^{CV} = X - \beta(Y - \mathbb{E}[Y])$$

This is mathematically correct because $Y - \mathbb{E}[Y]$ has expectation 0, so the estimator remains unbiased but variance is reduced if X and Y move together. In order to get the parameter σ_{eff}^2 with which to run the simulations, the pathwise total variance is used, defined as:

$$\sigma_{\text{eff}}^2 = \frac{1}{T} \int_0^T \sigma_t^2 dt \approx \frac{1}{N} \sum_{t=1}^N \sigma_t^2 \Delta t$$

Using this value, the conditional expectation proxy is calculated. This way, an estimate Y that is structurally similar to X , much less noisy and strongly correlated with X is obtained. This way, the final estimator is:

$$\hat{X} = \frac{1}{n} \sum (X_i - \beta(Y_i - \bar{Y}))$$

where:

- $\bar{Y}$ is the sample mean
- β is chosen to minimize variance

Furthermore, β will measure:

- If Y tracks X perfectly: large β , big variance reduction - If uncorrelated: $\beta \approx 0$, no benefit.

This is implemented in the repository as `path_dependent_cv(S0, K, T, r, rho, timesteps, n, exp)`. When compared to the previous Monte Carlo implementation, the reduction in variance of the estimator can be seen.

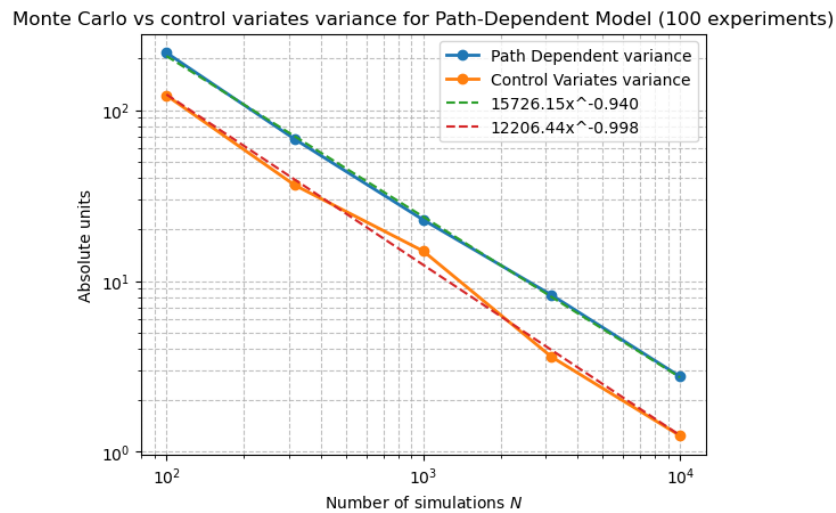


Figure 5.4: Path dependent MC vs Control Variates

6 Final findings

This repository documents some basic methods of simulating events using Monte Carlo techniques, using the Black Scholes model, due to its relevance in quantitative finance. The successive methods showcase the usefulness of variance reduction techniques, to get orders of magnitude improvements on well behaved problems. The last sections showcase how these methods can be adapted for either extreme situations, as well as more complex models. Path dependent options are only lightly introduced, but the computational complexity increase is noticed. Finally, some comparisons between the various methods can be done, to showcase the improvement over base Monte Carlo. Let efficiency be defined as $E = \frac{1}{\text{Time} \cdot \text{Var}}$, and relative efficiencies (compared to Monte Carlo) be computed. For moderate strike price and volatility estimates, the results are as following.

Table 6.1: Normalized efficiencies for moderate estimates

Method	Normalized Efficiency
Method	Normalized Efficiency
Monte Carlo	1
Antithetic valid	3
Stratified sampling	170
Sobol	335271

Note how the big increase for Sobol is due to the smaller variance, coupled with faster asymptotic reduction rate, which makes the difference much bigger as sample size increases. Similarly, when deep ITM estimates are used, the increase obtained by using Importance Sampling can be seen.

Table 6.2: Normalized efficiencies for deep ITM estimates

Method	Normalized Efficiency
Method	Normalized Efficiency
Monte Carlo	1
Antithetic valid	2
Stratified sampling	1
Sobol	284
Importance sampling	2276

License

This project is licensed under the GNU General Public License v3.0 or later (GPL-3.0-or-later). See the `LICENSE.md` file for details. If you use this repository, please cite as indicated in `citation.rff`.